In [155…
```
import pandas as pd
```

In [156…
```
import numpy as np
```

In [157…
```
df= pd.read_csv("Boston.csv")
```

In [158…
```
print(df.columns)
```

```
Index(['Unnamed: 0', 'crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis',
       'rad', 'tax', 'ptratio', 'black', 'lstat', 'medv', 'Price'],
      dtype='object')
```

In [159…
```
df
```

Out[159…

	Unnamed: 0	crim	zn	indus	chas	nox	rm	age	dis	rad	tax
0	1	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296
1	2	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242
2	3	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242
3	4	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222
4	5	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222
...	...	...	...	...	...	...	...	...	...	...	...
501	502	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273
502	503	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273
503	504	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273
504	505	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273
505	506	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273

506 rows × 16 columns

In [160…
```
import matplotlib.pyplot as plt
```

In [161…
```
import sklearn
```

In [162…
```
df["Price"]
```

Out[162…
```
0      417
1      274
2      136
3        0
4      450
      ...
501    501
502    259
503    149
504     86
505    593
Name: Price, Length: 506, dtype: int64
```

In [163…
```
y=df["Price"]
```

In [164…
```
y
```

Out[164…
```
0      417
1      274
2      136
3        0
4      450
      ...
501    501
502    259
503    149
504     86
505    593
Name: Price, Length: 506, dtype: int64
```

In [165…
```
x=df.drop("Price",axis=1)
```

In [166…
```
x
```

Out[166…

	Unnamed: 0	crim	zn	indus	chas	nox	rm	age	dis	rad	tax
0	1	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296
1	2	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242
2	3	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242
3	4	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222
4	5	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222
...	...	...	...	...	...	...	...	...	...	...	...
501	502	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273
502	503	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273
503	504	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273
504	505	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273
505	506	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273

506 rows × 15 columns

In [167…
```
from sklearn.model_selection import train_test_split
xtrain,xtest,ytrain,ytest = train_test_split(x,y,test_size=0.2, random_state=0
```

In [168…
```
from sklearn.linear_model import LinearRegression
lm=LinearRegression()
```

In [169…
```
model=lm.fit(xtrain,ytrain)
```

In [170…
```
ytrain_predict=lm.predict(xtrain)
ytest_predict=lm.predict(xtest)
```

In [171…
```
df
```

Out[171…

	Unnamed: 0	crim	zn	indus	chas	nox	rm	age	dis	rad	tax
0	1	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296
1	2	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242
2	3	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242
3	4	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222
4	5	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222
...	...	...	...	...	...	...	...	...	...	...	...
501	502	0.06263	0.0	11.93	0	0.573	6.593	69.1	2.4786	1	273
502	503	0.04527	0.0	11.93	0	0.573	6.120	76.7	2.2875	1	273
503	504	0.06076	0.0	11.93	0	0.573	6.976	91.0	2.1675	1	273
504	505	0.10959	0.0	11.93	0	0.573	6.794	89.3	2.3889	1	273
505	506	0.04741	0.0	11.93	0	0.573	6.030	80.8	2.5050	1	273

506 rows × 16 columns

In [172…

```
ytrain_predict
```

Out[172… array([274.35836632, 278.62290116, 266.87443951, 291.94083667,
347.87115093, 360.84925114, 280.72333756, 302.87325744,
294.58947194, 304.3790922 , 254.19306677, 314.43524057,
237.72580205, 254.12496699, 300.74786924, 266.43456399,
333.3426637 , 272.82178653, 323.79067766, 320.62213956,
258.28280392, 307.74821908, 296.2942476 , 213.95984266,
274.07651484, 251.54996376, 279.86363021, 262.19958038,
319.62146995, 299.4194479 , 296.06964034, 301.24204035,
314.49097707, 264.02492447, 300.47449399, 296.26040819,
271.12439229, 303.73026569, 320.06103321, 311.72767838,
284.96278091, 284.99324415, 304.34152891, 267.33787498,
315.50426978, 256.52274386, 321.49151689, 321.24910876,
292.89790696, 277.66817766, 238.18606342, 276.75836447,
253.05933861, 329.57630999, 209.95852567, 253.20055767,
289.76798492, 261.23875411, 318.79949648, 276.99386726,
287.62641338, 277.59655756, 226.36902967, 267.89933594,
290.2799798 , 275.66395297, 341.77213727, 342.35983724,
311.55970527, 323.62788052, 318.22752749, 312.57971431,
324.25900044, 324.05819495, 312.02730452, 293.09626147,
305.41228729, 274.13960063, 323.7341424 , 381.01663766,
278.8410626 , 306.93879965, 282.24779959, 289.40494494,
248.05843002, 292.97411507, 256.94835176, 325.36918843,
311.89401912, 302.7772996 , 291.35554651, 293.22443826,
315.83667766, 287.07932647, 294.82239875, 335.83013414,
314.12718673, 260.94808077, 270.60498921, 306.15421495,
304.26345055, 303.94649671, 285.50754006, 320.66726328,
317.91091021, 284.27313389, 318.40317896, 310.7462721 ,
281.02708212, 276.43682888, 336.90056049, 325.31305854,
288.7680481 , 308.40013249, 241.47336979, 289.67440281,
332.94911536, 261.98111686, 361.80741039, 257.07586225,
268.31672205, 331.06900186, 300.90381528, 317.87370795,
250.96513763, 319.67164945, 301.95936422, 288.68264409,
327.61959002, 292.13371277, 317.52965257, 266.91461797,
313.88280817, 300.65708156, 313.5434711 , 226.49395419,
314.23224193, 306.57347082, 288.1045714 , 297.64263879,
327.414175 , 265.65116266, 316.1316817 , 289.13937314,
292.93561126, 194.45882519, 287.96927716, 254.70151612,
321.87315563, 301.14295322, 347.97601533, 257.68777263,
269.22205812, 267.86504122, 306.32752601, 275.18536331,
254.82398785, 317.28863342, 293.27989785, 279.82749008,
254.14800945, 280.85069466, 299.91867408, 325.77063367,
257.27537486, 415.64542053, 278.11483407, 339.19263206,
263.78638082, 259.08414473, 296.9761331 , 261.97706984,
344.77593694, 280.0532602 , 324.85923541, 256.9163329 ,
263.00611086, 292.67853257, 284.74736271, 282.55094965,
285.35886741, 280.11532447, 326.53147349, 321.32437223,
247.99958256, 279.63286556, 283.56550591, 327.99352666,
299.60215929, 261.49970502, 275.92772774, 292.64884452,
270.15804278, 303.91810663, 267.98734939, 346.13221598,
307.51219614, 303.69365062, 292.35465474, 328.83917588,
299.5441688 , 264.04758212, 282.32464477, 288.71062094,
294.31510286, 358.57822482, 301.43130985, 282.99985355,
286.1249976 , 292.14528612, 266.34796767, 334.75408984,
294.55562721, 314.06314171, 350.54004083, 303.0000975 ,

```
308.81792914, 280.2535827 , 373.60158285, 256.48782688,
278.05042527, 311.92667185, 362.0725061 , 275.9359825 ,
266.71076559, 395.95241415, 262.56214636, 305.14749611,
334.28752964, 291.4925263 , 337.00919716, 257.68238101,
267.94067399, 266.56333792, 344.46015986, 271.36862814,
243.93640304, 220.43702608, 261.66033087, 264.22839969,
290.37684565, 287.84325027, 404.38953761, 329.0848178 ,
294.84300707, 270.13170233, 321.49071458, 273.99650539,
290.72552776, 285.5916128 , 287.34164464, 310.94248948,
282.41065355, 324.09535927, 280.99096222, 301.50207067,
298.05714134, 309.62849669, 277.36832072, 278.87656049,
278.91179852, 269.75472457, 240.62188101, 313.51840921,
323.19397129, 336.01134546, 191.09726788, 274.60557807,
266.04167018, 314.75946793, 262.75382942, 334.51258424,
254.80369069, 284.33543299, 328.89019197, 329.46087552,
336.97139892, 373.3119296 , 333.49134587, 275.49304091,
272.25417336, 275.64762483, 320.77887707, 301.87283393,
324.747162  , 278.94828101, 263.62051848, 308.49416518,
299.71601979, 323.37811451, 297.07277125, 295.15421517,
290.01977256, 213.67541111, 281.10537715, 306.61555228,
330.55110291, 253.78486155, 301.20938158, 283.20098634,
300.62077048, 301.55831283, 269.30731616, 251.08890379,
318.70403629, 266.83430681, 313.85267902, 268.00519245,
282.13901223, 285.82799077, 281.62877988, 277.78560779,
221.82534432, 309.88472452, 340.42382006, 323.27565144,
309.4656168 , 351.18634942, 303.77879742, 293.58346616,
278.68522354, 322.43851198, 331.06027717, 281.0732542 ,
297.97287853, 329.3426814 , 283.36673427, 281.5282712 ,
306.61578385, 315.37868689, 290.40810888, 288.49258694,
325.76058293, 305.00351378, 237.61062621, 316.88773552,
299.77557928, 334.86320793, 288.39250728, 330.58675431,
314.98569938, 257.68582885, 293.77874245, 256.67299476,
286.79397181, 320.70662533, 250.18640621, 339.2762219 ,
270.14518011, 310.19148057, 270.31448352, 236.98455409,
271.95957291, 277.99105373, 298.14337518, 312.51128402,
287.36334323, 311.53196858, 342.38119085, 323.98101813,
304.26959079, 302.05486152, 286.66775064, 317.14113381,
276.92523346, 283.15831737, 320.38945819, 301.26842549,
276.30441024, 305.55583027, 303.77335033, 327.45109835,
309.443293  , 356.50261277, 361.46130363, 291.14805663,
271.79318096, 273.66327607, 310.79362155, 270.16630628,
297.34275758, 324.97318465, 308.84240992, 307.68331572,
305.30165596, 275.41552875, 312.0900844 , 291.04980444,
305.04057551, 278.41752533, 295.22038643, 296.50034376,
275.61898085, 283.37836215, 241.11244809, 267.09755081,
335.65251215, 289.51675462, 294.08748594, 280.92953968,
316.5100227 , 321.75467704, 238.87849334, 311.79272116])
```

In [173…

```
ytest_predict
```

Out[173…
```
array([334.31357452, 329.58686569, 263.09260784, 360.76930439,
       272.05797555, 294.09896687, 333.67119602, 278.92762366,
       332.41432236, 299.90388577, 406.02852329, 308.87239673,
       319.66748761, 350.63844907, 268.89135184, 311.88975712,
       318.59388682, 313.31938847, 323.05352047, 305.44829555,
       281.16646252, 307.29303225, 291.54478735, 295.81759289,
       289.94609998, 398.55613444, 270.45701822, 326.81400232,
       260.91308564, 297.38365619, 315.6657393 , 327.19395086,
       258.88877948, 281.64230096, 309.40185851, 337.43306203,
       324.83375262, 338.12096403, 319.75099333, 292.55445143,
       277.47947969, 274.66805676, 326.28058441, 328.48469782,
       321.44190522, 267.6410403 , 248.83726135, 310.62844554,
       307.97896669, 283.50744045, 322.62504223, 306.04293394,
       298.69457949, 227.16817923, 269.31874247, 292.84818704,
       297.86786353, 273.98483339, 412.05764215, 244.67749039,
       305.62633781, 280.92525751, 299.1868991 , 318.98403506,
       339.1739511 , 344.99685615, 305.41181024, 249.48819498,
       300.14506304, 288.88704839, 275.83598147, 287.6894001 ,
       331.48208137, 309.55916552, 313.52639588, 332.19753342,
       253.47227692, 266.55665782, 303.12549736, 306.08649893,
       290.15757008, 260.02925198, 353.87608965, 258.22433877,
       241.6608661 , 286.4866457 , 296.50791939, 353.47176695,
       326.7051058 , 316.15365473, 299.41343185, 306.99658095,
       300.12772707, 304.78991055, 288.99335362, 273.16390071,
       324.11794446, 300.87317426, 316.78796682, 265.68263377,
       276.45973124, 283.65019141])
```

In [174…
```
df1=pd.DataFrame({'Actual':ytrain,'Predicted': ytrain_predict})
df2=pd.DataFrame({'Actual':ytest, 'Predicted': ytest_predict})
```

In [175…
```
df1
```

Out[175…

	Actual	Predicted
220	592	274.358366
71	359	278.622901
240	466	266.874440
6	344	291.940837
417	69	347.871151
...	...	...
323	109	280.929540
192	560	316.510023
117	397	321.754677
47	164	238.878493
172	423	311.792721

404 rows × 2 columns

In [176…

```
df2
```

Out[176…

	Actual	Predicted
329	598	334.313575
371	130	329.586866
219	388	263.092608
403	347	360.769304
78	208	272.057976
...	...	...
56	149	300.873174
455	431	316.787967
60	174	265.682634
213	210	276.459731
108	82	283.650191

102 rows × 2 columns

In [177…

```
from sklearn.metrics import mean_squared_error
```

In [178…

```
m1 = mean_squared_error(ytrain, ytrain_predict)
```

In [179…
```
m2 = mean_squared_error(ytest, ytest_predict)
```

In [180…
```
print("MSE (Train):", m1)
```

```
MSE (Train): 32170.627222303963
```

In [181…
```
print("MSE (Test):", m2)
```

```
MSE (Test): 28814.173891222625
```

In [182…
```
m1
```

Out[182…
```
32170.627222303963
```

In [183…
```
m2
```

Out[183…
```
28814.173891222625
```

In [184…
```
from sklearn.metrics import r2_score
```

In [185…
```
r1 = r2_score(ytrain, ytrain_predict)
```

In [186…
```
r2 = r2_score(ytest, ytest_predict)
```

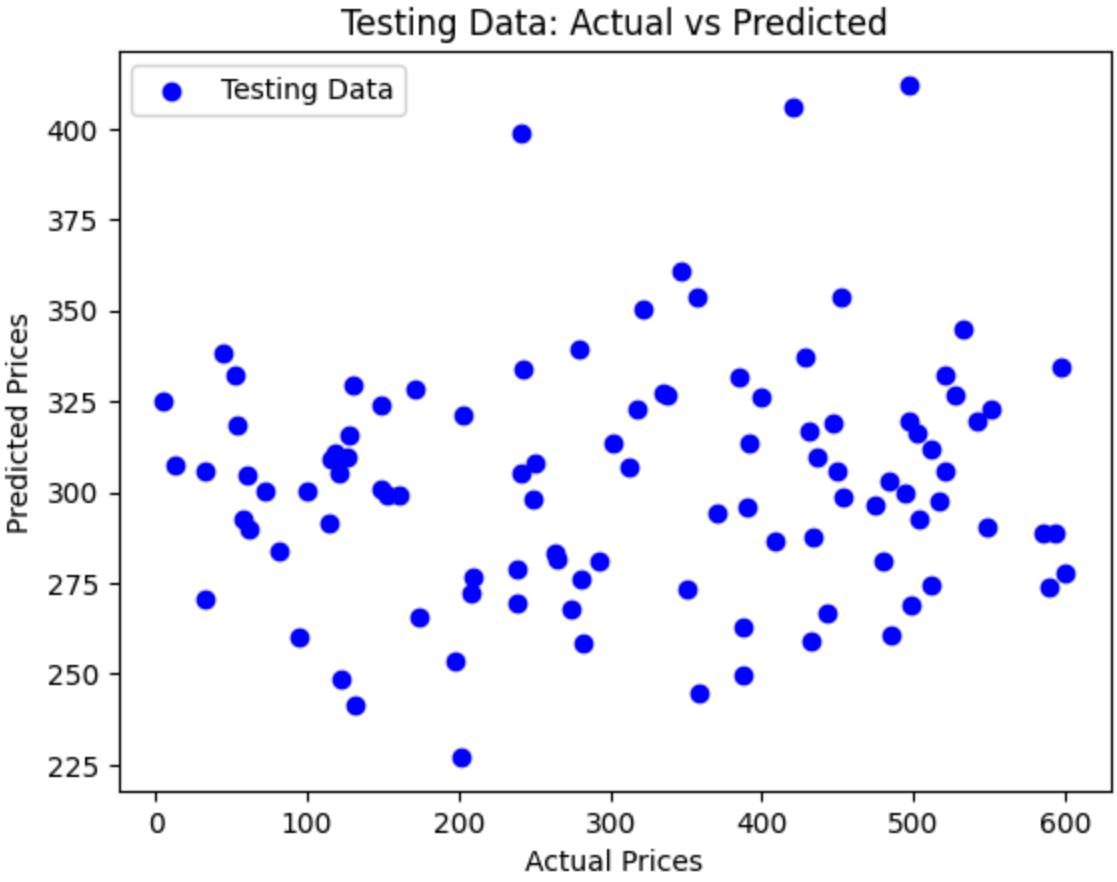
In [187…
```
print("R² (Train):", r1)
```

```
R² (Train): 0.02892093407178431
```

In [188…
```
print("R² (Test):", r2)
```

```
R² (Test): -0.003882390260427915
```

In [189…
```
# Plot Testing Data
plt.scatter(ytest, ytest_predict, c="blue", marker="o", label="Testing Data")
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Testing Data: Actual vs Predicted")
plt.legend()
plt.show()
```



In [190…

```
# Plot Training Data
plt.scatter(ytrain, ytrain_predict, c="green", marker="o", label="Training Dat
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Training Data: Actual vs Predicted")
plt.legend()
plt.show()
```


Training Data: Actual vs Predicted
Training Data
Predicted Prices
Actual Prices
400
350
300
250
200
0
100
200
300
400
500
600

In [ ]: